

Function Evaluation Grading Report

Executive summary

Two 1,000-sample prediction sets were graded against the same `evals/function/v1` test split under **Jac 0.36.1**:

- 08**: Qwen3-Coder-30B-A3B with the experiment 08 LoRA adapter.
- base**: the same base model without the adapter.

The adapter produces a large improvement in output validity and Jac compatibility, but neither run produces a successful task:

Metric	08	base	Change (08 - base)
Samples	1,000	1,000	—
Task success / pass@1	0.0%	0.0%	0.0 pp
Jac check rate	73.7%	5.9%	+67.8 pp
Hidden behavior-test rate	0.0%	0.0%	0.0 pp
Contract rate	99.7%	59.2%	+40.5 pp
Extraction failures	2 (0.2%)	129 (12.9%)	-127 / -12.7 pp
Check failures	261 (26.1%)	812 (81.2%)	-551 / -55.1 pp
Test failures	737 (73.7%)	59 (5.9%)	+678 / +67.8 pp
Infrastructure errors	0	0	—

Main conclusion: experiment 08 solves much of the syntax/extraction problem and allows 737 samples to reach hidden tests, but behavior correctness is currently the limiting factor. The base run mostly fails before execution, so its 0% success is dominated by invalid Jac output rather than tested behavior.

Evaluation setup

- Split: `test`
- Tasks: 1,000 total; 500 completion and 500 translation
- Samples: 1 per task (`k=1`)

- Jac version: 0.36.1 (Linux x86_64)
- Workers: 4
- Per-stage timeout: 900 seconds
- Problems hash: 5170c9c08e0b973baf07f1400928eea322d377fad6786c09ce882cda5c3c5576
- Grader: fn_eval_preds/eval/eval_jac.py
- Results: fn_eval_preds/out/08/ and fn_eval_preds/out/base/

A task requires successful checking, passing all hidden tests, and satisfying its feature contract. All tasks in this run contain behavior tests; therefore, compile-only results do not affect the reported outcome.

Results by task type

Completion track

Metric	08	base
Samples	500	500
Check pass	307 (61.4%)	59 (11.8%)
Test executed	307 (61.4%)	59 (11.8%)
Test pass	0 (0.0%)	0 (0.0%)
Status: check_fail	191	317
Status: extract_fail	2	124
Status: test_fail	307	59
Contract pass	497/500 (99.4%)*	271/500 (54.2%)*

Translation track

Metric	08	base
Samples	500	500
Check pass	430 (86.0%)	0 (0.0%)
Test executed	430 (86.0%)	0 (0.0%)
Test pass	0 (0.0%)	0 (0.0%)

Metric	08	base
Status: check_fail	70	495
Status: extract_fail	0	5
Status: test_fail	430	0
Contract pass	500/500 (100.0%)*	321/500 (64.2%)*

* Contract-pass counts are calculated from the per-row field; extraction-failure rows do not contain a contract result and are treated as false by the aggregate grader.

Paired comparison

The two runs use the same task IDs, so their statuses can be compared task by task.

- On completion tasks, **08 made 258 cases checkable that were not checkable in base.**
- On translation tasks, **08 made 430 cases checkable that were not checkable in base.**
- Overall, 08 enabled hidden-test execution on **688 tasks that base could not execute.**
- Base had no checkable translation sample; 08 reached hidden tests for 430 translation samples.

This confirms that the adapter's primary measured benefit is conversion into valid Jac, not yet functional correctness.

Failure analysis

08

The 08 run has 263 samples that never reached tests:

- 261 check failures
- 2 extraction failures

The 737 test failures show that the generated code passes the checker but does not pass the hidden behavior suite. Frequent diagnostic signals in the captured test output include:

Signal / error code	Rows or occurrences
E1053 type assignment/argument mismatch	219 test-failure rows containing the code
E5092 unsupported native lowering	124
E5043 bytecode compilation failure	120

Signal / error code	Rows or occurrences
E5090 unsupported native call pathway	29
“preferred native but did not lower”	545 rows
PostgreSQL startup text in captured test output	126 rows

The native-lowering messages are emitted while running the hidden tests and do not by themselves prove that the model code is semantically wrong. They do show that the test path often falls back from the preferred native pathway. The PostgreSQL startup text is also environment-like evidence; the grader nevertheless recorded these rows as `test_fail`, not `infra_error`, so the reported 0% behavior rate should be interpreted with this caveat.

The 261 check failures include recurring Jac conversion issues such as:

- Python control-flow syntax or indentation retained in the output.
- Missing braces/semicolons or malformed function structure.
- Inconsistent static types and missing return paths.
- Duplicated docstrings or duplicated declarations in completion outputs.

base

The base run has 941 samples that never reached tests:

- 812 check failures
- 129 extraction failures

Translation is the weakest area: 495/500 translations fail checking and 5/500 fail extraction. Common checker diagnostics include:

Signal / error code	Rows or occurrences
E0005 unexpected token	478 check-failure rows containing the code; 732 total code occurrences
E0030 unexpected semicolon/module-level structure	120 rows; 120 total occurrences
E0002 missing semicolon	38 rows; 57 total occurrences
E1004 possible implicit <code>None</code> return	41 rows; 41 total occurrences
Incomplete fenced <code>jac</code> extraction	129 samples

Signal / error code	Rows or occurrences
---------------------	---------------------

Representative failures contain Python syntax such as colon-based function definitions, indentation-based blocks, Python comprehensions, and Python `return` statements inside Jac-braced functions. This explains why almost all base samples fail before behavioral testing.

Static diagnostics

These are diagnostics only, not an aggregate idiomaticity score.

Diagnostic	08	base
Typed function-definition rate	99.30%	43.17%
No Python syntax	99.90%	68.08%
No Python imports	99.90%	99.89%
No dynamic types	91.28%	94.14%
Mean source lines	11.76	13.73
Mean line length	34.19	33.84
Mean lines over 100 chars	0.162	0.163
Mean maximum brace depth	2.18	1.75

The adapter improves Jac-specific structure and typed declarations substantially. The slight decrease in the “no dynamic types” diagnostic suggests that stronger typing should be checked without sacrificing support for heterogeneous Python inputs.

Runtime and reproducibility notes

- Both summaries report `complete: true` and `infra_errors: 0`.
- The runs used the same problems hash and Jac version, so the aggregate comparison is internally consistent.
- The 08 run took 2,449.9 seconds (about 40.8 minutes); base took 527.6 seconds (about 8.8 minutes). This difference is expected because 08 reached hidden tests for 737 samples versus 59 for base.
- The original provenance records generation under Jac 0.16.1 and this grading cross-check under Jac 0.36.1. Results in this report are only the Jac 0.36.1 results.

Recommended next steps

1. **Prioritize hidden-test pass rate, not only check rate.** Use the 737 executable 08 samples as the main debugging corpus.
2. **Separate environment failures from model failures.** Re-run or isolate the 126 08 rows containing PostgreSQL startup errors before drawing conclusions about those tasks.
3. **Cluster test failures by exact hidden-test assertion and compiler diagnostic.** Address E1053, unsupported lowering (E5092/E5043/E5090), and runtime assertion failures separately.
4. **Improve completion-specific repair.** The completion track has substantially more check failures than translation and shows duplicated docstrings/declarations and malformed block boundaries.
5. **Retain extraction and syntax gates in future evaluations.** They clearly expose the base model's dominant failure mode and the adapter's strongest gain.

Source artifacts

- `fn_eval_preds/out/08/summary.json`
- `fn_eval_preds/out/08/results.jsonl`
- `fn_eval_preds/out/08/grade.log`
- `fn_eval_preds/out/base/summary.json`
- `fn_eval_preds/out/base/results.jsonl`
- `fn_eval_preds/out/base/grade.log`
- `fn_eval_preds/README.md`
- `fn_eval_preds/provenance.json`